

HACKATHON SUBMISSION

AI Change Request Analyzer

Turn ambiguous change requests into engineering decisions.

Submitted by Shivani Patel

Team / Hackathon *[add your team and hackathon name here]*

Date September 2026

Repository / Source Provided separately (source code archive)

20

FEATURE MODULES

400+

AUTOMATED TESTS

4

SWAPPABLE AI PROVIDERS

0

DOCKER CONTAINERS NEEDED

1. Problem Statement

Every software change starts life as a short, plain-language request: a sentence or two describing what someone wants changed. Turning that sentence into an actual engineering decision — what does this really require, what will it touch, what could go wrong, how risky is it, who needs to sign off, how much effort is this — is normally manual, slow, and inconsistent. Two different reviewers can read the same change request and reach very different conclusions about its scope and risk, and nothing about the process leaves a clear, structured trail of what was assessed, by whom, and against which version of the request.

Teams need a way to turn an ambiguous written request into a structured, defensible engineering assessment — automatically, consistently, and without losing the human judgment that ultimately has to approve it.

2. Solution Overview

The **AI Change Request Analyzer** takes a plain-language change request and reads it the way a senior engineer would: it extracts concrete requirements, classifies the change, assesses its impact across seven fixed lenses (business, technical, customer, operational, security, data, performance), identifies risks, runs a structured 9-category security review, finds the parts of the actual codebase it's likely to touch, retrieves relevant excerpts from the team's own project documentation as grounding evidence, estimates complexity and effort, writes test cases and an implementation plan, and recommends what kind of human approval it should need.

None of that is a black box: every finding is tagged with the AI's own certainty and confidence, and every one of them can be reviewed, confirmed, or overridden by a person. On top of that AI core sits a full enterprise change-management workflow — editing with real version history, a status lifecycle, role-based team assignments, multi-type approvals with notifications, threaded comments with @mentions, and an append-only audit trail that nothing in the app can edit after the fact. The AI recommends; only a human ever approves.

3. Key Features

AI-Powered Analysis — requirement extraction, classification, impact assessment, risk scoring, a structured security review, complexity/effort estimation, missing-information detection, generated test cases, and a generated implementation plan, from one written change request.

Repository Intelligence — scans and indexes the project's own source files (imports, functions, classes, API routes, database/config references) and matches a change request against real files with honest, hedged confidence — never a claimed certainty the evidence doesn't support.

Knowledge Base & RAG — upload project documentation; the AI retrieves genuinely relevant excerpts (a real similarity cutoff, not just "the closest thing on file") as grounding context, with a Source/Section citation on every piece of evidence.

Version-Aware Everything — editing a change request creates a new version and flags every earlier analysis, risk, requirement, and repository finding as outdated until it's re-analyzed. Nothing is ever silently mixed across versions, including in the generated PDF report.

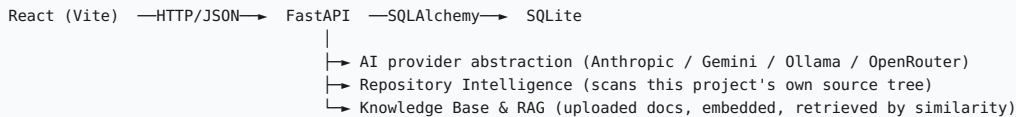
Enterprise Workflow — a full status lifecycle, role-based assignments, multi-type approvals (Technical / Security / Product / Engineering Manager / Director / QA / DBA / Release / General), threaded comments with @mentions, and notifications for every one of them — including a direct way for a person to answer the AI's own clarification questions when it flags missing information.

Administration — user management and roles, a role × capability permissions matrix, admin-configurable approval rules, and a system-level audit log — all enforced on the backend, not just hidden in the UI.

Audit & Reporting — an append-only activity trail on every change request (no API can edit or delete a past entry), plus a version-aware, CAB-ready PDF report assembled entirely from already-persisted data.

Analytics — executive, workflow, risk, approval-bottleneck, change, workload, and AI-recommendation metrics, computed from real database records, never invented.

4. Architecture



One backend process, one frontend process, one SQLite file. No message queue, no background worker, no second database, no container orchestration — the guiding constraint throughout was to stay exactly as complex as the problem requires and no further.

Frontend

React 19 + Vite, deliberately with no client-side router — a single piece of state (`activeView`) decides which page renders. Each feature area is its own page component, talking to the backend through small fetch-based API wrapper modules that mirror the backend's own one-router-per-feature layout.

Backend

FastAPI, one router module per feature area. Business logic lives in a dedicated service layer, not the API layer, so the actual rules (valid status transitions, required approvals, version diffing, similarity ranking) each live in one independently readable place. Authentication is a signed JWT; passwords are hashed with PBKDF2.

Database

A single SQLite file, created and kept up to date automatically on every startup — no separate migration framework, since a hackathon-scale schema doesn't need one. Every enumerated field (status, priority, risk category) is a real Python Enum, so an invalid value is rejected at the database layer.

AI Engine

One small interface (`complete()` , `is_configured()`) with one implementation per provider — Anthropic, Gemini, Ollama, OpenRouter — selected at runtime by an environment variable. Every AI finding carries its own certainty (known / inferred / unknown) and confidence score, set by the AI itself and never inferred after the fact. A malformed or incomplete AI response is rejected with a clear error rather than partially persisted, and there is no "demo mode" that fabricates a response if the provider is unavailable.

Workflow, Approvals & Audit

A single status enum (Draft → Submitted → Pending Analysis → Analyzed → Under Review → Approved → Implementation Planned → In Progress → Implemented → Validated → Closed, with Cancelled reachable from most states) enforced centrally, so an illegal transition is never even offered as an option. Editing a change request never overwrites history — it bumps a version, snapshots the change, and flags every downstream AI artifact outdated until re-analyzed. Approvals are their own model, tagged to a specific person for a specific approval type; the AI only ever recommends which types are needed, a human always decides who to ask. Two separate append-only audit trails exist — one per change request, one for system-level administrative actions — and neither can be edited or deleted through any API route, only inserted into.

5. Tech Stack

Layer	Technology
Backend	Python, FastAPI, SQLAlchemy 2.0, Pydantic, SQLite, JWT auth (PyJWT), PBKDF2 password hashing, fpdf2 for PDF reports
Frontend	React 19, Vite — no client-side router, a single authenticated shell switches between pages by local state
AI	Provider-agnostic abstraction supporting Anthropic (Claude), Google Gemini, Ollama (fully local, no API key), and OpenRouter — swapping providers is one environment variable
Testing	pytest, 400+ automated backend tests against a throwaway database created fresh for every test run
Deliberately not used	Docker, a message queue, a background worker, a second database, a client-side routing library, Kubernetes — kept as simple as the problem allows

Why this stack

Every dependency choice favored the smallest thing that solves the actual problem correctly, over what's fashionable. SQLite instead of Postgres because this is a single-process app with no concurrent- write contention to design around; no client-side router because the app has no deep-linkable URL structure that would benefit from one; a provider-agnostic AI layer instead of hard-coding one vendor because the requirement was "an AI backend," not "an Anthropic-specific integration."

6. Setup & Installation

Prerequisites

Python 3.10+, Node.js 18+ and npm.

1. Backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
```

Open `backend\.env` and fill in at least a `SECRET_KEY` (generate one with `python -c "import secrets; print(secrets.token_hex(32))"`) and one AI provider key (Gemini has a free tier with no billing; Ollama runs entirely locally with no key at all).

2. Frontend

```
cd frontend
npm install
copy .env.example .env
```

3. Start both servers

```
Terminal 1 (backend): cd backend && python -m uvicorn app.main:app --reload
Terminal 2 (frontend): cd frontend && npm run dev
```

Backend runs at `http://localhost:8000` (interactive API docs at `/docs`); frontend at `http://localhost:5173`. Database tables are created automatically on first startup — no separate migration step.

4. Optional demo data

```
cd backend
python -m app.database.seed_demo
```

Loads five demo accounts (all on the fictional `alight.com` domain, password `AlightDemo123!`) and a flagship change request for a live walkthrough — see Section 7.

5. Run the test suite

```
cd backend
python -m pytest ../tests
```

Runs 400+ automated backend tests against a throwaway SQLite database created fresh for the run — this never touches the live demo database, so testing is always safe alongside a live demo.

7. Demo Walkthrough

A condensed script for presenting the app end to end, built around one flagship change request — **"Add OTP-based authentication for customers"** — seeded already created but deliberately left unanalyzed, so the AI analysis step is a real, live call, not a replay.

Demo accounts

Name	Email	Role
Ava Chen	admin@alight.com	Admin
Jordan Blake	manager@alight.com	Manager
Liam Carter	engineer@alight.com	Engineer (drives the demo)
Priya Nair	security@alight.com	Security Reviewer
Sofia Reyes	requester@alight.com	Requester

All local-only demo credentials, password `AlightDemo123!` for every account.

Walkthrough steps

- Log in as the Engineer and open the flagship change request — status "Pending Analysis," no analysis yet.
- Click **Analyze Change** — a real, live AI call. Once complete, walk through the Analysis Dashboard's tabs: Requirements, Risks, Security (9-category structured review), Impact, Repository (real files from this actual codebase), Complexity & Effort, Test Cases, and Implementation Plan.
- Assign a Technical Lead and a Security Reviewer, then request a Security approval — note the AI's own recommended approval type shown above the list; the app never requests one automatically.
- Log in as the Security Reviewer, see the notification, and respond **Changes Requested** with a comment.
- Log back in as the Engineer, edit the change request to address the feedback, and save — the version badge increments (v1 → v2), and an amber "AI analysis outdated" banner appears.
- Click **Re-analyze** — a second live AI call, re-scoring risk against the new version.
- Log in as the Security Reviewer again and respond **Approved**.
- As the Engineer, walk the status forward: Approved → Implementation Planned → In Progress → Implemented → Validated → Closed — each transition gated by a single central rule, so an illegal jump is never even offered.
- Download the final PDF report — assembled entirely from already-persisted data, including the full audit history, and clearly labeled if the underlying analysis is ever outdated.

If the AI provider is unavailable during a live demo: Analyze/Re-analyze fails with a clear, specific error and nothing is written to the database — there is no "demo mode" that fabricates a response. Every other step (workflow, approvals, versioning, audit, reporting) works independently of the AI and can carry the rest of the demo.

8. Closing Note

"Turn ambiguous change requests into engineering decisions." The point of this system is that it does more than summarize a paragraph of text back to the reader — it extracts real requirements, scores real risk, runs a real security review against a fixed framework, matches against the project's actual source files, estimates effort, writes test cases and an implementation plan, and keeps every one of those artifacts honestly tied to the exact version of the change request they were computed against — while every decision that actually matters (who approves, whether to proceed, what "done" means) stays with a person.

Full technical documentation (module-by-module build log, complete source code, and the full presentation script) is provided separately alongside this submission.